databricks 10.delta_lake_demo

(https://databricks.com)

1. Write data to delta lake (managed table)
2. Write data to delta lake (external table)
3. Read data from delta lake (Table)
4. Read data from delta lake (File)

2

```
%sql
CREATE DATABASE IF NOT EXISTS f1_demo
LOCATION '/mnt/formula1dl1204/demo'
```

OK

3

```
results_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/results.json")
```

4

```
results_df.write.format("delta").mode("overwrite").saveAsTable("f1_demo.results_managed")
```

5

```
%sql
SELECT * FROM f1_demo.results_managed;
```

Table

	constructorId	driverId	fastestLap	fastestLapSpeed	fastestLapTime	grid	laps
1	131	1	44	207.235	1:34.015	2	56
2	9	830	41	208.984	1:33.228	1	56
3	131	822	56	211.566	1:32.090	3	56
4	1	846	38	206.398	1:34.396	7	56
5	9	815	44	207.334	1:33.970	0	56
6	6	844	39	205.112	1:34.988	4	56
7	1	817	36	205.233	1:34.932	6	56
8	6	832	48	206.151	1:34.509	8	56
9	213	852	38	205.603	1:34.761	13	56
10	117	840	31	205.378	1:34.865	10	56
11	51	8	45	204.672	1:35.192	14	56
12	51	841	32	204.823	1:35.122	12	55
13	214	839	33	204.548	1:35.250	16	55
14	3	847	40	205.008	1:35.036	15	55

20 rows

6

```
results_df.write.format("delta").mode("overwrite").save("/mnt/formula1dl1204/demo/results_external")
```

7

```
%sql
CREATE TABLE f1_demo.results_external
USING DELTA
LOCATION '/mnt/formula1dl1204/demo/results_external'
```

OK

8

```
%sql
SELECT * FROM f1_demo.results_external
```

Table

	constructorId	driverId	fastestLap	fastestLapSpeed	fastestLapTime	grid	laps
1	131	1	44	207.235	1:34.015	2	56
2	9	830	41	208.984	1:33.228	1	56
3	131	822	56	211.566	1:32.090	3	56
4	1	846	38	206.398	1:34.396	7	56
5	9	815	44	207.334	1:33.970	0	56
6	6	844	39	205.112	1:34.988	4	56
7	1	817	36	205.233	1:34.932	6	56
8	6	832	48	206.151	1:34.509	8	56
9	213	852	38	205.603	1:34.761	13	56
10	117	840	31	205.378	1:34.865	10	56
11	51	8	45	204.672	1:35.192	14	56
12	51	841	32	204.823	1:35.122	12	55
13	214	839	33	204.548	1:35.250	16	55
14	3	847	40	205.008	1:35.036	15	55

20 rows

9

```
results_external_df = spark.read.format("delta").load("/mnt/formula1dl1204/demo/results_external")
```

10

```
display(results_external_df)
```

Table

	constructorId	driverId	fastestLap	fastestLapSpeed	fastestLapTime	grid	laps
1	131	1	44	207.235	1:34.015	2	56
2	9	830	41	208.984	1:33.228	1	56
3	131	822	56	211.566	1:32.090	3	56
4	1	846	38	206.398	1:34.396	7	56
5	9	815	44	207.334	1:33.970	0	56
6	6	844	39	205.112	1:34.988	4	56
7	1	817	36	205.233	1:34.932	6	56
8	6	832	48	206.151	1:34.509	8	56
9	213	852	38	205.603	1:34.761	13	56
10	117	840	31	205.378	1:34.865	10	56
11	51	8	45	204.672	1:35.192	14	56
12	51	841	32	204.823	1:35.122	12	55
13	214	839	33	204.548	1:35.250	16	55
14	3	847	40	205.008	1:35.036	15	55

20 rows

11

```
results_df.write.format("delta").mode("overwrite").partitionBy("constructorId").saveAsTable("f1_demo.results_partitioned")
```

12

```
%sql
SHOW PARTITIONS f1_demo.results_partitioned
```

Table

	constructorId
1	1
2	3
3	6
4	9
5	51
6	131
7	117
8	214
9	210
10	213

10 rows

1. Update Delta Table
2. Delete From Delta Table

14

```
%sql
SELECT * FROM f1_demo.results_managed;
```

15

```
%sql
UPDATE f1_demo.results_managed
  SET points = 11 - position
WHERE position <= 10
```

16

```
%sql
SELECT * FROM f1_demo.results_managed;
```

17

```
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/mnt/formula1dl1204/demo/results_managed")

deltaTable.update("position <= 10", { "points": "21 - position" } )
```

18

```
%sql
SELECT * FROM f1_demo.results_managed;
```

19

```
%sql
DELETE FROM f1_demo.results_managed
WHERE position > 10;
```

20

```
%sql
SELECT * FROM f1_demo.results_managed;
```

21

```
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/mnt/formula1dl1204/demo/results_managed")

deltaTable.delete("points = 0")
```

22

```
%sql
SELECT * FROM f1_demo.results_managed;
```

Upsert using merge

24

```
drivers_day1_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/drivers.json") \
.filter("driverId <= 10") \
.select("driverId", "dob", "name.forename", "name.surname")
```

25

```
display(drivers_day1_df)
```

26

```
drivers_day1_df.createOrReplaceTempView("drivers_day1")
```

27

```
from pyspark.sql.functions import upper

drivers_day2_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/drivers.json") \
.filter("driverId BETWEEN 6 AND 15") \
.select("driverId", "dob", upper("name.forename").alias("forename"), upper("name.surname").alias("surname"))
```

28

```
drivers_day2_df.createOrReplaceTempView("drivers_day2")
```

29

```
display(drivers_day2_df)
```

30

```
from pyspark.sql.functions import upper

drivers_day3_df = spark.read \
.option("inferSchema", True) \
.json("/mnt/formula1dl1204/raw/2021-03-28/drivers.json") \
.filter("driverId BETWEEN 1 AND 5 OR driverId BETWEEN 16 AND 20") \
.select("driverId", "dob", upper("name.forename").alias("forename"), upper("name.surname").alias("surname"))
```

31

```
%sql
CREATE TABLE IF NOT EXISTS f1_demo.drivers_merge (
driverId INT,
dob DATE,
forename STRING,
surname STRING,
createdDate DATE,
updatedDate DATE
)
USING DELTA
```

Day1

33

```
%sql
MERGE INTO f1_demo.drivers_merge tgt
USING drivers_day1 upd
ON tgt.driverId = upd.driverId
WHEN MATCHED THEN
  UPDATE SET tgt.dob = upd.dob,
             tgt.forename = upd.forename,
             tgt.surname = upd.surname,
             tgt.updatedDate = current_timestamp
WHEN NOT MATCHED
  THEN INSERT (driverId, dob, forename,surname,createdDate ) VALUES (driverId, dob, forename,surname, current_timestamp)
```

34

```
%sql SELECT * FROM f1_demo.drivers_merge;
```

Day 2

36

```
%sql
MERGE INTO f1_demo.drivers_merge tgt
USING drivers_day2 upd
ON tgt.driverId = upd.driverId
WHEN MATCHED THEN
  UPDATE SET tgt.dob = upd.dob,
             tgt.forename = upd.forename,
             tgt.surname = upd.surname,
             tgt.updatedDate = current_timestamp
WHEN NOT MATCHED
  THEN INSERT (driverId, dob, forename,surname,createdDate ) VALUES (driverId, dob, forename,surname, current_timestamp)
```

37

```
%sql SELECT * FROM f1_demo.drivers_merge;
```

Day 3

39

```
from pyspark.sql.functions import current_timestamp
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/mnt/formula1dl1204/demo/drivers_merge")

deltaTable.alias("tgt").merge(
    drivers_day3_df.alias("upd"),
    "tgt.driverId = upd.driverId") \
  .whenMatchedUpdate(set = { "dob" : "upd.dob", "forename" : "upd.forename", "surname" : "upd.surname", "updatedDate": "current_tim
  .whenNotMatchedInsert(values =
    {
      "driverId": "upd.driverId",
      "dob": "upd.dob",
      "forename" : "upd.forename",
      "surname" : "upd.surname",
      "createdDate": "current_timestamp()"
    }
  ) \
  .execute()
```

40

```
%sql SELECT * FROM f1_demo.drivers_merge;
```

1. History & Versioning
2. Time Travel
3. Vaccum

42

```
%sql
DESC HISTORY f1_demo.drivers_merge
```

43

```
%sql
SELECT * FROM f1_demo.drivers_merge VERSION AS OF 2;
```

44

```
%sql
SELECT * FROM f1_demo.drivers_merge TIMESTAMP AS OF '2021-06-23T15:40:33.000+0000';
```

45

```
df = spark.read.format("delta").option("timestampAsOf", '2021-06-23T15:40:33.000+0000').load("/mnt/formula1dl1204/demo/drivers_merg
```

46

```
display(df)
```

47

```
%sql
VACUUM f1_demo.drivers_merge
```

48

```
%sql
SELECT * FROM f1_demo.drivers_merge TIMESTAMP AS OF '2021-06-23T15:40:33.000+0000';
```

49

```
%sql
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM f1_demo.drivers_merge RETAIN 0 HOURS
```

50

```
%sql
SELECT * FROM f1_demo.drivers_merge TIMESTAMP AS OF '2021-06-23T15:40:33.000+0000';
```

51

```
%sql
SELECT * FROM f1_demo.drivers_merge
```

52

```
%sql
DESC HISTORY f1_demo.drivers_merge;
```

53

```
%sql
DELETE FROM f1_demo.drivers_merge WHERE driverId = 1;
```

54

```
%sql
SELECT * FROM f1_demo.drivers_merge VERSION AS OF 3;
```

55

```
%sql
MERGE INTO f1_demo.drivers_merge tgt
USING f1_demo.drivers_merge VERSION AS OF 3 src
   ON (tgt.driverId = src.driverId)
WHEN NOT MATCHED THEN
   INSERT *
```

56

```
%sql DESC HISTORY f1_demo.drivers_merge
```

57

```
%sql
SELECT * FROM f1_demo.drivers_merge
```

Transaction Logs

59

```
%sql
CREATE TABLE IF NOT EXISTS f1_demo.drivers_txn (
driverId INT,
dob DATE,
forename STRING,
surname STRING,
createdDate DATE,
updatedDate DATE
)
USING DELTA
```

60

```
%sql
DESC HISTORY f1_demo.drivers_txn
```

61

```
%sql
INSERT INTO f1_demo.drivers_txn
SELECT * FROM f1_demo.drivers_merge
WHERE driverId = 1;
```

62

```
%sql
DESC HISTORY f1_demo.drivers_txn
```

63

```
%sql
INSERT INTO f1_demo.drivers_txn
SELECT * FROM f1_demo.drivers_merge
WHERE driverId = 2;
```

64

```
%sql
DELETE FROM  f1_demo.drivers_txn
WHERE driverId = 1;
```

65

```
for driver_id in range(3, 20):
  spark.sql(f"""INSERT INTO f1_demo.drivers_txn
                SELECT * FROM f1_demo.drivers_merge
                WHERE driverId = {driver_id}""")
```

66

```
%sql
INSERT INTO f1_demo.drivers_txn
SELECT * FROM f1_demo.drivers_merge;
```

Convert Parquet to Delta

68

```
%sql
CREATE TABLE IF NOT EXISTS f1_demo.drivers_convert_to_delta (
driverId INT,
dob DATE,
forename STRING,
surname STRING,
createdDate DATE,
updatedDate DATE
)
USING PARQUET
```

69

```
%sql
INSERT INTO f1_demo.drivers_convert_to_delta
SELECT * FROM f1_demo.drivers_merge
```

70

```
%sql
CONVERT TO DELTA f1_demo.drivers_convert_to_delta
```

71

```
df = spark.table("f1_demo.drivers_convert_to_delta")
```

72

```
df.write.format("parquet").save("/mnt/formula1dl1204/demo/drivers_convert_to_delta_new")
```

73

```
%sql
CONVERT TO DELTA parquet.`/mnt/formula1dl1204/demo/drivers_convert_to_delta_new`
```

74